Differing behaviour in qml.Optimizers for QAOA tutorial

https://github.com/PennyLaneAI/pennylane/issues/788

ROW 121

Failed to reproduce 600-qubit VQE because of being out of memory #61

New issue

Closed

royess opened on Sep 16, 2022

I tried to reproduce the 600-qubit VQE mentioned in the whitepaper by running a script slightly modified from examples/vqe_extra_mpo.py. Modifications are:

1. Only do one forward evaluation;
2. Use CPU instead of GPU.

I use a machine with 36 cores and 251GB memory run the script. The backend is set as tensorflow. But the job is killed because of being out of memory, while the whitepaper shows that a 40GB GPU is enough for a same job.

▶ Version information.

▶ The script.

refraction-ray on Sep 17, 2022 Contributor

Is the out of memory error raised at jit staging time or runtime?
Also, one should set a rather large time buffer for the cotengra optimizer to search for the better tensor network contraction path, max_time=360 is not enough. The path searching for the n=600 problem may takes one hour or so to get an reasonable path. The way to check whether the path finder is good enough is focusing at the summary information of contraction path, the WRITE metric is equivalent to the memory consumption 2^WRITE up to some constant overhead

royess on Sep 17, 2022 Contributor Author

> Is the out of memory error raised at jit staging time or runtime?

In my script, I suppose jit is not enabled. So the out of memory error should be at runtime.

And thanks for your suggestions, I will have another try by setting a larger `max_time`.

royess on Sep 18, 2022 Contributor Author

The memory cost is reduced to ~30GB when I use a `max_time=3600` for finding the contraction path. Many thanks!

👍 1

royess closed this as completed on Sep 18, 2022

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

Differing behaviour in qml.Optimizers for QAOA tutorial #788

New issue

Closed

milanleonard opened on Sep 8, 2020 · edited by milanleonard — Edits

I'm trying to benchmark different optimizers for this tutorial (and in general) https://pennylane.ai/qml/demos/tutorial_qaoa_maxcut.html. When I change the optimizer from `qml.AdagradOptimizer` to `qml.QNGOptimizer` I have an issue with the `metric_tensor_fn` where it says it requires a single QNode or VQEObject, however the objective function is not a `qml.qnode`.

```
ValueError                                Traceback (most recent call last)
 in
     43 # perform qaoa on our graph with p=1,2 and
     44 # keep the bitstring sample lists
---> 45 bitstrings1 = qaoa_maxcut(n_layers=1)[1]
     46 bitstrings2 = qaoa_maxcut(n_layers=2)[1]

 in qaoa_maxcut(n_layers)
     22     steps = 30
     23     for i in range(steps):
---> 24         params = opt.step(objective, params)
     25         if (i + 1) % 5 == 0:
     26             print("Objective after step {:5d}: {: .7f}".format(i + 1, -objective(params)))

/usr/local/anaconda3/envs/main/lib/python3.7/site-packages/pennylane/optimize/qng.py in step(self, qnode,
x, recompute_tensor, metric_tensor_fn)
    174         if not hasattr(qnode, "metric_tensor") and not metric_tensor_fn:
    175             raise ValueError(
--> 176                 "The objective function must either be encoded as a single QNode or "
    177                 "a VQECost object for the natural gradient to be automatically computed. "
    178                 "Otherwise, metric_tensor_fn must be explicitly provided to the optimizer."

ValueError: The objective function must either be encoded as a single QNode or a VQECost object for the n
atural gradient to be automatically computed. Otherwise, metric_tensor_fn must be explicitly provided to
 the optimizer.
```

Trying naively to use the `qml.qnode` wrapper I get a separate error

```
Trusted  Jupyter Server: local  Python

kwargs)
     88         # the template decorator and the operation recorder.
     89         for context in cls._active_contexts:
---> 90             context._append(obj, **kwargs)  # pylint: disable=protected-access
     91
     92     @abc.abstractmethod

/usr/local/anaconda3/envs/main/lib/python3.7/site-packages/pennylane/qnodes/base.py in _append(self, ob
j)
    382             if self.obs_queue:
    383                 raise QuantumFunctionError(
--> 384                     "State preparations and gates must precede measured observables."
    385                 )
    386             self.queue.append(obj)

QuantumFunctionError: State preparations and gates must precede measured observables.
```

Any ideas for how to get around this highly appreciated!
[meta p.s.] is this type of question better placed here or on the forum?

nquesada on Sep 8, 2020 · edited by co9olguy — Edits — Contributor

Hi **@milanleonard** . Thanks for the report. In the note in the documentation of https://pennylane.readthedocs.io/en/stable/code/api/pennylane.QNGOptimizer.html you can read a bit more about how to utilise the quantum natural gradient functionality. It seems like the cost function in the QAOA tutorial is neither a single `QNode` nor a `VQECost`.

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

co9olguy on Sep 8, 2020 · edited by co9olguy — Edits — Member

As a bit more explanation, the QNG Optimizer explicitly looks at the structure of a variational circuit in order to determine its parameter update. Thus it only knows what to do with a single QNode. If you have a hybrid computation, there is then a "mixing" of parameters, some coming from the variational circuit, and some (potentially) coming from classical parts of your model, or other QNodes. The QNG optimizer has no update rule for such "extra-circuit" parameters.

milanleonard on Sep 9, 2020 · edited by milanleonard — Edits — Author

@co9olguy Thanks for the extra info. There's no extra-circuit parameters per se but I'm still struggling to remap this circuit into a single QNode object (which should definitely be possible). In particular I keep running into the issue that I can't act on the computation of the measurement within the QNode, since (I think) it is lazily evaluated when QNode.evaluate() (or `__call__`) is executed, which means I can't write something like

```
neg_obj = sum([-0.5*(1-qml.expval(qml.Hermitian(pauli_z_2, wires=edge))) for edge in edges])
        return neg_obj
```

And if I try to run this I get an error `TypeError: unsupported operand type(s) for -: 'int' and 'Hermitian'`. I cannot figure out how to get around writing this as a single QNode object, in that it naturally feels like it wants me to qml.map over the template for the collection of expval observables, but I can't do that if I want to use QNG. Is there any way to force those expvals to return the values, or do you have any suggestions as to how I might (even if it's janky) get around this.

Sorry that I didn't phrase my original question well enough!

josh146 on Sep 9, 2020 — Member

Hi @milanleonard, depending on how you've structured your cost function, there is a potential work around.

Say you have a cost function consisting of multiple QNodes, and each QNode has

1. the same circuit structure,
2. the same parameter input, and
3. there is no 'mixing' of the parameters with classical processing (as @co9olguy has mentioned).

Then, you can take advantage of the fact that `QNGOptimizer.step` accepts an optional `metric_tensor_fn` argument, where you can pass the explicit metric tensor function you wish the optimizer to use. This avoids the need for the optimizer to 'inspect' the cost function directly to determine the metric tensor function itself.

For example, consider a very basic VQE example:

```
def circuit(params, **kwargs):
    qml.RX(params[0], wires=0)
    qml.RY(params[1], wires=1)
    qml.CNOT(wires=[0, 1])

dev = qml.device("default.qubit", wires=2)
obs_list = [qml.PauliZ(0) @ qml.PauliZ(1), qml.PauliZ(0) @ qml.PauliZ(1)]
qnodes = qml.map(circuit, obs_list, dev)
```

Our cost function will simply be some linear combination of the QNodes:

```
def cost(params):
    return np.array([0.2, -0.5]) @ qnodes(params)
```

Since this cost function satisfies our requires above, we can simply pass the metric tensor function of the first QNode to the optimizer:

```
params = np.array([0.1, 0.2])
opt = qml.QNGOptimizer(0.01)
for i in range(100):
    params = opt.step(cost, params, metric_tensor_fn=qnodes[0].metric_tensor)
```

Goodr

Say you have a cost function consisting of multiple QNodes, and each QNode has

1. the same circuit structure,
2. the same parameter input, and
3. there is no 'mixing' of the parameters with classical processing (as **@co9olguy** has mentioned).

Then, you can take advantage of the fact that `QNGOptimizer.step` accepts an optional `metric_tensor_fn` argument, where you can pass the explicit metric tensor function you wish the optimizer to use. This avoids the need for the optimizer to 'inspect' the cost function directly to determine the metric tensor function itself.

For example, consider a very basic VQE example:

```
def circuit(params, **kwargs):
    qml.RX(params[0], wires=0)
    qml.RY(params[1], wires=1)
    qml.CNOT(wires=[0, 1])

dev = qml.device("default.qubit", wires=2)
obs_list = [qml.PauliZ(0) @ qml.PauliZ(1), qml.PauliZ(0) @ qml.PauliZ(1)]
qnodes = qml.map(circuit, obs_list, dev)
```

Our cost function will simply be some linear combination of the QNodes:

```
def cost(params):
    return np.array([0.2, -0.5]) @ qnodes(params)
```

Since this cost function satisfies our requires above, we can simply pass the metric tensor function of the first QNode to the optimizer:

```
params = np.array([0.1, 0.2])
opt = qml.QNGOptimizer(0.01)
for i in range(100):
    params = opt.step(cost, params, metric_tensor_fn=qnodes[0].metric_tensor)
```

milanleonard on Sep 9, 2020 Author

Thanks **@josh146** that's perfect. Looks like the custom metric tensor argument has come in handy a few times

milanleonard closed this as completed on Sep 9, 2020